EXP:6 - GRADIENT DESCENT METHOD- STOCHASTIC GRADIENT DESCENT ALGORITHM

AIM:

To iteratively adjust the parameters of a model in order to minimize the error between the predicted and actual values by using the Stochastic Gradient Descent (SGD) algorithm. SGD updates the model parameters based on each training example, allowing for faster convergence and handling larger datasets efficiently compared to batch gradient descent.

PROCEDURE:

1. Initialization:

- We initialize the weights to zeros.
- X is the feature matrix, and y is the target vector.
- learning_rate controls the step size of each update.
- epochs is the number of iterations over the entire dataset.

2. Gradient Computation:

- For each sample in the dataset, we compute the gradient of the loss with respect to the
- weights.
- The gradient is calculated as (prediction - target) * feature.
- Weights Update:
- We update the weights by moving them in the opposite direction of the gradient, scaled
- by the learning rate.

Note:

● The data X is a matrix where each row is a sample and each column is a feature.

● The target values y are reshaped to ensure they are in the correct format for the computations.

● In the example, we generate some synthetic data for linear regression with added noise.

CODE:

```
import numpy as np


def stochastic_gradient_descent(X, y, learning_rate=0.01, epochs=1000):


    n_samples, n_features = X.shape

    weights = np.zeros(n_features)


    for epoch in range(epochs):

        for i in range(n_samples):
```

```
        gradient = (np.dot(X[i], weights) - y[i]) * X[i]
        weights -= learning_rate * gradient

    return weights


# New dataset (House Price Prediction)
np.random.seed(0)
X = 2 * np.random.rand(100, 1)  # area in 1000 sq ft
y = 50 + 20 * X + np.random.randn(100, 1)  # price in lakhs

X_b = np.c_[np.ones((100, 1)), X]  # bias term
y = y.ravel()

learning_rate = 0.01
epochs = 1000

weights = stochastic_gradient_descent(X_b, y, learning_rate, epochs)

print("Learned Weights:", weights)
```

Output:
Learned Weights: [50.16464189 19.90371342]

Result:

The given linear regression model was trained successfully using Stochastic Gradient Descent in Python. The optimal weights for predicting house prices were obtained.